1 Policies (read this first)

Deadline and late window. Lab3 is due by 11:59pm on Friday, November 13. Late submissions are accepted until 8:00am on Sunday, November 15 with an automatic 20% penalty. Nothing is accepted after that, for any reason. We always grade the last submission on Canvas, so a resubmission during the late window gets the penalty. Solutions are posted on Sunday, November 15.

AI policy. AI use (ChatGPT, Claude, Gemini, …) is allowed in this lab but must be disclosed: say in your README which tool you used and what for. Undisclosed AI use is an academic integrity violation. You must be able to explain, defend, and modify every line you submit, and the teaching team may quiz you on it.

Lab Check 3. In your lab section during week 8 (November 16 to 20), first 20 minutes: a short paper check on the Lab3 material, closed book, no devices, no AI. It is easy if you did and understood your own lab. No make-ups.

2 Goal

In Lab3, you will create and query a view, and you will create an index and see how it changes the way a query is executed. You will also be required to combine new data (as explained below) into one of the tables. Finally, you will need to add some new constraints to the database and do some unit testing to see that the constraints are followed.

1. Create and query a view
2. Create an index, and compare a query before and after creating it
3. Perform SQL to "combine data" from two tables
4. Add foreign key constraints
5. Add general constraints
6. Write unit tests for constraints

Under Resources -> Lab3, we will provide you with data that you can load into your tables so that you can test the results of your queries. Testing can prove that a query is wrong, but not that it is right, so be careful. We will not give you the results of these queries on the load data; you should be able to figure out the results on that data yourselves. You can also test your queries on your own data. In the "real world", you have to make and check your own tests.

3 Working with a View

3.1 Create PostgreSQL Schema Lab3

You will create a Lab3 schema to set apart the database tables created in this lab from the tables you will create in future, as well as from tables (and other objects) in the default (public) schema. In PostgreSQL, a database can have more than one schema; see the PostgreSQL documentation on schemas for more details. You create the Lab3 schema as follows:

```
CREATE SCHEMA Lab3;
```

Now that you have created the schema, you want to set Lab3 to be your default schema when you use psql. If you do not set Lab3 as the default schema, then you will have to qualify your table names with the schema name (e.g., Lab3.Contract). To set the default schema, you modify your search path.

```
ALTER ROLE cse180 SET SEARCH_PATH to Lab3;
```

You will need to log out and log back in to the server for this default schema change to take effect. (Type \q, then run the docker exec command from the Setup Guide again.) (Students often forget to do this.)

You do not have to include the CREATE SCHEMA or ALTER ROLE statements in your solution.

3.2 Create tables

Under Resources -> Lab3, we have provided a create script named *create_lab3.sql* and a load script named *load_lab3.sql* that loads tuples into the tables of the schema. You must run both *create_lab3.sql* and *load_lab3.sql* before you run the parts of Lab3 that are described below. (As in Lab1 and Lab2, keep the files in your CSE180-lab folder, which psql sees as /CSE180-lab.)

```
\i /CSE180-lab/create_lab3.sql
\i /CSE180-lab/load_lab3.sql
```

In the *create_lab3.sql* file that we have provided under Resources -> Lab3, the first 7 tables are similar to the tables that were in our Lab1 *create_lab1.sql* solution; the NULL and UNIQUE constraints from Lab2 are not included. Moreover, *create_lab3.sql* is missing the Foreign Key constraints on Contract and PayloadIntegration that were in *create_lab1.sql* (the Foreign Keys on Mission and MissionCrew are kept). You will create new variations of those constraints in later sections.

In practice, primary keys, unique constraints, and other constraints are almost always entered when tables are created, not added later. *create_lab3.sql* handles some constraints for you, but you will be adding some additional constraints to these tables in Lab3, as described below.

Note also that there is an additional table, UpdateContract, in the *create_lab3.sql* file that has some of the attributes that are in the Contract table. We will say more about UpdateContract below.

3.2.1 Create a view

A contract in the Contract table has a price that the customer pays to fly its payloads on a mission. Many missions are "rideshare" missions that carry payloads for several contracts at once. We want to identify those missions, specifically, the missions that have 2 or more contracts. These "rideshare missions" are an important source of revenue for the launch company.

Create a view called RideshareMissionView, which identifies missions that have 2 or more contracts. The attributes in the view should be:

- missionID
- rocketType
- totalContracts (the count of contracts for this mission)
- totalRevenue (the sum of all price values for this mission's contracts)

Only include a mission in RideshareMissionView if it has 2 or more contracts.

No duplicates should appear in your view. Save the script for creating that view in a file called *createview.sql*.

3.2.2 Query a View

You will write a script called *queryview.sql* that contains a query which we will refer to as the RideshareMissionSummary query. (You do not actually need to name the query, or create another view for this.) The RideshareMissionSummary query uses RideshareMissionView, and (possibly) some other tables.

In addition to writing the RideshareMissionSummary query, you must also include some comments in the *queryview.sql* script; we will describe those necessary comments below.

Write and run a SQL query which calculates the average revenue per contract for each rideshare

mission. The attributes in your result should be missionID, rocketType, totalContracts, and avgRevenuePerContract (calculated as totalRevenue divided by totalContracts).

Run this file with that view, then write the results of the RideshareMissionSummary query in a comment. The format of that comment is not important; the comment just has to have all the right information in it.

Next, write a SQL statement that deletes the following tuple from the Contract table:

- The tuple whose Primary Key (contractID) is 209.

Run the RideshareMissionSummary query once again after that deletion. Write the output of the query in a second comment. Do you get a different answer? Think about why the results changed.

You need to submit a script named *queryview.sql* containing:

1. Your RideshareMissionSummary SQL query.
2. A comment with the output of that query on the load data before the deletion.
3. A SQL statement which performs the deletion described above.
4. Repeat your RideshareMissionSummary SQL query.
5. A second comment with the output of that query after the deletion.

3.3 Create and Use an Index

Important: Before running this section, recreate the Lab3 schema using the *create_lab3.sql* script, and load the data using the script *load_lab3.sql*. Then also run the script *load_lab3_large.sql*, which we have also provided under Resources -> Lab3. That way, any database changes that you have done for the view section will not propagate to this part of Lab3.

```
\i /CSE180-lab/create_lab3.sql

\i /CSE180-lab/load_lab3.sql

\i /CSE180-lab/load_lab3_large.sql
```

load_lab3_large.sql adds 200,000 more contracts (with contractID values 100001 to 300000) to the Contract table, so that Contract is large enough for an index to make a difference. Only about 2% of these new contracts have priority 'H'. The script ends with `ANALYZE Contract;`, which updates the statistics that PostgreSQL uses to decide how to execute a query.

Indexes are data structures used by the database to improve query performance. Locating the tuples in the Contract table for a particular priority and price might be slow, if the database system has to search the entire Contract table (if its size was very large). In this section, you will run the same query before and after you create an index, and compare both the results and the time that the query takes.

You will write a script called *queryindex.sql* that contains a query which we will refer to as the PremiumContract query. A *premium contract* is a contract whose priority is 'H' and whose price is at least 90000.00. For each mission that has at least one premium contract, the PremiumContract query outputs the missionID and rocketType of that mission, numPremium (the number of premium contracts for that mission), and avgPremiumPrice (the average price of those premium contracts). No duplicates should appear in your result. Your result should be sorted by missionID, so that it is easy to compare the outputs before and after the index is created.

To see how PostgreSQL executes a query and how long it takes, put `EXPLAIN ANALYZE` in front of the query. EXPLAIN ANALYZE runs the query, but instead of the query result, it shows the *query plan* (how PostgreSQL executed the query), and its last line shows the Execution Time. In the plan, look for the line that mentions contract. `Seq Scan on contract` (or `Parallel Seq Scan on contract`) means that PostgreSQL read every tuple of Contract; a line that mentions `contractindex` (for

example, `Bitmap Index Scan on contractindex`) means that PostgreSQL used your index. Please refer to the documentation of PostgreSQL on EXPLAIN that is at https://www.postgresql.org/docs/15/sql-explain.html.

Your Execution Times depend on your computer, so they will not be the same as other students' times, and the first run of a query can be slower than later runs. We will not grade the time values themselves.

You need to submit a script named *queryindex.sql* containing:

1. Your PremiumContract SQL query.
2. A comment with the output of that query, before the index is created.
3. EXPLAIN ANALYZE of your PremiumContract query, and a comment that says how Contract was scanned, and what the Execution Time was.
4. A SQL statement that creates an index named ContractIndex over the priority and price columns (in that order) of the Contract table.
5. Repeat your PremiumContract SQL query, and a comment with its output after the index is created.
6. Repeat EXPLAIN ANALYZE of your PremiumContract query, and a comment that says how Contract was scanned, and what the Execution Time was.
7. A last comment that answers these questions in one or two sentences: Did the result of the query change after the index was created? Did the Execution Time change? Why?

If you are interested, try this too (it is not graded, and you should not include it in *queryindex.sql*): recreate the schema and load only *load_lab3.sql*, create ContractIndex, and run EXPLAIN on the PremiumContract query. Does PostgreSQL use the index on this small Contract table? Why not?

Note: *load_lab3_large.sql* changes the data in the Contract table. Before you work on any other section of Lab3 (including going back to the view section), recreate the Lab3 schema using *create_lab3.sql* and load the data using *load_lab3.sql*.

3.4 Combine Data

Important: Before running this section, recreate the Lab3 schema using the *create_lab3.sql* script, and load the data using the script *load_lab3.sql*. That way, any database changes that you have done for previous sections will not propagate to this part of Lab3.

We have a new table here. Its primary key is underlined:

- UpdateContract(<u>contractID</u>, priority, price)

Write a file, *combine.sql*, that will do the following:

For each tuple in UpdateContract, there might already be a tuple in the Contract table that has the same Primary Key (that is, the same value for contractID). If there is not a tuple in Contract with the same Primary Key, then this is a new contract that should be inserted into the Contract table. If there already is a tuple in Contract with that Primary Key, then this is an update of information about that contract. So here are the effects that your SQL statements should have:

1. If there already is a tuple in the Contract table which has that contractID, then update the tuple in Contract which has that contractID using the priority and price values in the UpdateContract tuple. Also, update the paymentMethod to 'WireTransfer'.
2. If there is not already a tuple in the Contract table which has that contractID, then you should insert a tuple into the Contract table which has the contractID, priority, and price values that are in the UpdateContract tuple. Also, the missionID should be set to NULL, the signedDate should be set to CURRENT_DATE, and the paymentMethod should be set to 'WireTransfer'.

Your *combine.sql* may have multiple SQL statements in it.

3.5 Add Foreign Key Constraints

Important: Before running this section, recreate the Lab3 schema using the *create_lab3.sql* script, and load the data using the script *load_lab3.sql*. That way, any database changes that you have done for previous sections will not propagate to this part of Lab3.

Here is a description of the Foreign Keys that you need to add for this assignment. (Foreign Key Constraints are also referred to as Referential Integrity constraints.) The *create_lab3.sql* file that we have provided for Lab3 includes only some of the Referential Integrity constraints that were in the Lab1 solution, but you are asked to use ALTER to add additional constraints to the Lab3 schema.

The load data that you have been given should not cause any errors when you add these constraints. Just add the constraints listed below, exactly as described, even if you think that additional Referential Integrity constraints should exist. Note that (for example) when we say that each mission (missionID) that appears in the Contract table must appear in the Mission table, that means that the missionID attribute of the Contract table is a Foreign Key referring to the Primary Key of the Mission table (which also is missionID).

1. Each mission (missionID) that appears in the Contract table must appear in the Mission table as a Primary Key (missionID). If a tuple in the Mission table is deleted, then the missionID in all affected Contract tuples should be set to NULL. If the Primary Key (missionID) of a tuple in the Mission table is updated, then all Contract tuples for that updated mission should also be updated to the new missionID value.
2. Each payload (payloadID) that appears in the PayloadIntegration table must appear in the Payload table as a Primary Key (payloadID). If a tuple in the Payload table is deleted and there are PayloadIntegration tuples which correspond to that payloadID, then that payload tuple deletion should be rejected. If the Primary Key (payloadID) of a tuple in the Payload table is updated, then all PayloadIntegration tuples for that updated payload should also be updated to the new payloadID value.
3. Each contract (contractID) that appears in the PayloadIntegration table must appear in the Contract table as a Primary Key (contractID). If a tuple in the Contract table is deleted and there are PayloadIntegration tuples that correspond to that contract, then all PayloadIntegration tuples which correspond to the deleted contract should also be deleted. If a Primary Key in the Contract table is updated, then all PayloadIntegration tuples which correspond to that contract's Primary Key should also be updated the same way.

Write commands to add foreign key constraints in the same order that the foreign keys are described above. Your foreign key constraints should all have names, but you may choose any names that you like. Save your commands to the file *foreign.sql*.

3.6 Add General Constraints

The general constraints for Lab3, which should be written as CHECK constraints, are:

1. In LaunchSite, numLaunchPads must be greater than zero. This constraint should be named siteLaunchPadsPositive.
2. In Mission, the value of outcome must be 'S', 'F', 'P', or NULL. This constraint should be named validMissionOutcome.
3. In Contract, if price is NULL, then paymentMethod must also be NULL. This constraint should be named ifNullPriceThenNullMethod.

Write commands to add general constraints in the order the constraints are described above, and save

your commands to the file *general.sql*. Remember that values TRUE and UNKNOWN are okay for a CHECK constraint, but FALSE is not.

3.7 Write Unit Tests

Unit tests are important for verifying that your constraints are working as you expect. We will require tests for just a few common cases, but there are many more unit tests that are possible. The unit tests you write need to do what is described below on the specific database instance that contains the Lab3 load data.

For each of the 3 foreign key constraints specified in Section 3.5, write one unit test:

- An INSERT command that violates the foreign key constraint (and elicits an error). You must violate that specific foreign key constraint, not any other constraint. **Be careful:** when testing a foreign key, make sure all other columns in your INSERT use values that satisfy every other constraint on the table (including primary keys and other foreign keys), so that the only violation is the one you intend to test. (Both Foreign Keys 2 and 3 are on PayloadIntegration.)

Also, for each of the 3 general constraints, write 2 unit tests. This means that you will write 2 tests for the first general constraint, followed by 2 tests for the second general constraint, followed by 2 tests for the third general constraint.

- An UPDATE command that meets the constraint.
- An UPDATE command that violates the constraint (and elicits an error).

Save these 3 + 6 = 9 unit tests in the order specified above in the file *unittests.sql*.

4 Testing

Before you submit, login to your database via psql and execute the provided database creation and load scripts, and then test your seven scripts (*createview.sql, queryview.sql, queryindex.sql, combine.sql, foreign.sql, general.sql, unittests.sql*). Note that there are three sections in this document (all labeled Important) where you are told to recreate the schema and reload the data before running that section, so that updates you performed earlier will not affect that section. Please be sure that you follow these directions, since your answers may be incorrect if you do not. (The commands below assume that you have run `\cd /CSE180-lab` first.)

```
-- ============================================================
-- Section 3.2: Create and Query a View
-- ============================================================
\i create_lab3.sql
\i load_lab3.sql
\i createview.sql
\i queryview.sql
-- ============================================================
-- Section 3.3: Create and Use an Index
-- ============================================================
\i create_lab3.sql
\i load_lab3.sql
\i load_lab3_large.sql
\i queryindex.sql
-- ============================================================
-- Section 3.4: Combine Data
-- ============================================================
\i create_lab3.sql
\i load_lab3.sql
```

```
\i combine.sql
-- ============================================================
-- Section 3.5 + 3.6 + 3.7: Foreign Keys, General Constraints, Unit Tests
-- ============================================================
\i create_lab3.sql
\i load_lab3.sql
\i foreign.sql
\i general.sql
\i unittests.sql
```

5 DISTINCT Policy

In this course, DISTINCT must be used exactly when it is needed. All three of these cases lose points:

1. The query does NOT ask for duplicate removal, but you add DISTINCT anyway — this changes the required output and will be penalized.
2. The query asks for no duplicates, and your query could return duplicates without DISTINCT (e.g., the output attributes don't include a key of the rows that are joined) — omitting DISTINCT will produce incorrect results and will be penalized.
3. The query asks for no duplicates, but your query cannot return duplicates anyway (e.g., the output includes the Primary Key of the only table in FROM, or GROUP BY already produces one row per output value) — adding DISTINCT here is unnecessary and **will also be penalized**.

So "no duplicates should appear in your result" does not mean "always write DISTINCT". For each query, ask yourself: can two different combinations of rows that satisfy my WHERE clause produce the same output row? If they can, you need DISTINCT (or GROUP BY). If they cannot, leave DISTINCT out. For example, `SELECT astronautID, astronautName FROM Astronaut WHERE agency = 'NASA'` cannot return duplicates, because astronautID is the Primary Key of Astronaut.

6 Submitting

1. Save your scripts indicated above as:

   - createview.sql
   - queryview.sql
   - queryindex.sql
   - combine.sql
   - foreign.sql
   - general.sql
   - unittests.sql

   Don't forget to include the comments in *queryview.sql* and *queryindex.sql* (lines that start with two hyphens are interpreted as comment lines). Put any other information for the Graders in a separate README file that you may submit. If you used an AI tool, the README is where you disclose it (see Policies).

2. Zip the files to a single file with the name Lab3_XXXXXXX.zip where XXXXXXX is your 7-digit student ID. For example, if a student's ID is 1234567, then the file that this student submits for Lab3 should be named Lab3_1234567.zip.

   To create the zip file you can use the Unix command:

   ```
   zip Lab3_1234567 createview.sql queryview.sql queryindex.sql combine.sql foreign.sql
   general.sql unittests.sql README
   ```

(Leave README out of the command if you don't have one. Of course, you use your own student ID, not 1234567.) Upload the zip file and, in the same Canvas submission, also upload each file individually (the seven .sql files, and README if you have one). This lets us grade you even if the zip is broken. A submission with only a zip that turns out to be empty or wrong cannot be fixed after the solutions are posted.

3. Be sure to follow directions about Academic Integrity that are in the Syllabus. If you have any questions about those directions, please speak to the instructor as soon as possible.
4. Lab3 is due by 11:59pm on Friday, November 13. See Policies at the top of this document for the late window. Always check to make sure that your submission is on Canvas, and that you've submitted the correct files.